Homework #1
EC535, Spring 2025
Assigned: January 23, Thursday -- **Due:** January 30, Thursday, 8:00 pm

---

This homework is a warm-up exercise of **C programming** under Linux environment. Use the **PHO 307 (or 305)** lab computers for this homework OR access the engineering grid:
eng-grid.bu.edu / eng-grid2.bu.edu via **ssh**.

Note that if you are off-campus, you will need to use **VPN to access the grid.*

0. In-class component (January 23):
Update the C code you are given (hw1intro.c) such that the "myNewThread" function asks the user to enter an integer and then prints out this integer in decimal, binary, and hexadecimal form.

Submit to ece535submit@gmail.com

with subject line:

yourBUusername_hw1inclass

---

1. Many of the future assignments will require basic knowledge of make. A good reference for writing the Makefile can be found at: http://www.gnu.org/software/make/manual/make.html. Study the reference, and create a bookmark in your browser to the link for future reference.

2. Write a C program which reads an input file containing a list of unsigned **little-endian** 32-bit decimal integers (one integer per line), and prints three columns to an output file. The first column should be the original decimal integer. The second column should have a hexadecimal value, corresponding to the number in the input file, but this time converted into **big-endian format** (consider *byte-endianness*, not bit-endianness)**.** The third column should show the ASCII representation (4 letters, each letter is 8-bits long) of each big-endian number, encrypted using XOR encryption with a *user-provided* 4-letter key.

   Your program should consist three files: **main.c, bits.c** and **bits.h**. You main.c file contains the main function and processes file I/O. Your bits.* files should define two functions:
   ```
   <type> ConvertEndianness(<type>);
   <type> EncryptXOR(<type>);
   ```

   You should determine which data types to use for the functions above. The first function converts a little-endian number into big-endian. The second function performs encryption. The resulting executable should be named "**BitApp**". BitApp takes the name of the input file as its first command line argument, name of the output file as the second argument, and the 4-letter key as its third argument, such as:
   ```
   ./BitApp input.txt output.txt qemu
   ```

   You can have other additional functions defined in your code, but you should make sure everything is properly defined and implemented in bits.c/.h.

3. Implement a linked list to maintain a prioritized list of the items (i.e., each item may include the input number, hexadecimal representation, and the ASCII representation; you can determine the contents yourself). In the

output file, the information printed should be *sorted based on the ASCII representation (ascending order)*. This feature can be implemented via arrays as well; however, implementing a linked list is required for this homework for full credit. Check out online links or printed resources on when to use linked lists vs. arrays.

Your linked list implementation must be contained in your source files bits.c/.h. Note that a simple linked list implementation (without all possible features of a linked list) is sufficient for this task.

4. Write a Makefile to compile your program from parts 2 and 3. The Makefile should compile **only the necessary files** after an update of the source code. For example, if only main.c is updated, the Makefile should only compile main.c but not bits.c; but if bits.h is updated, all .c files that include it should be recompiled. The default (first) rule should produce the "**BitApp**" executable. Provide a *clean* rule in the Makefile which will remove all object files (and executables) generated by running make.

   Use the **"gcc"** command to compile your program. If you are not familiar with gcc, type "man gcc" to learn its command line options.

   Please write your code cleanly and include comments. Please include your name as a comment at the beginning of each file.

Reminders

- You should work completely on your own. You can discuss general ideas with others, but **you should never look at other people's code and never share your code. This course uses automated software for assignment similarity checking.**
- **No late submissions are allowed**

Submission Instructions

- Make sure to follow the guidelines for function names, executable names, folder naming conventions, etc. not to lose any points. This class makes use of scripts and software while grading. So it is essential for everyone to follow the given guidelines.
- Put your **three source files (main.c, bits.c** and **bits.h) and the Makefile** in a folder called YourBUusername_HW1. Do not include any other files – they will not be graded. Submit a single compressed archive, YourBUusername_HW1.zip, on GradeScope.
- You can submit multiple times. However, keep in mind that we will always grade your latest submission and will discard any earlier submission from the same student.
- If you are coding in an environment other than the eng-grid or lab machines, make sure your code compiles and works before submitting. Assignments that do not compile WILL NOT be graded.
- Tentative rubric:
  - Makefile: 20 points
  - Main.c and I/O handling: 20 points
  - Bit manipulation (endianness, xor cipher) and type conversions: 40 points
  - Correct implementation and use of linked lists: 20 points